

[Airbus Amber]

3DX One Platform

Parametric Data Model

DRAFT

Design Patterns

AIRBUS

Summary

- Context & Objectives
- Data Model Principles (Reminders)
- Parametric Data Model Design Rules
- Move to new Design Patterns
- Rules to merge datamodel instantiation BL
- Design Pattern BL implementation
- Value improvement - Technical Artefacts reduction
- Technical artefacts governance
- Limitations

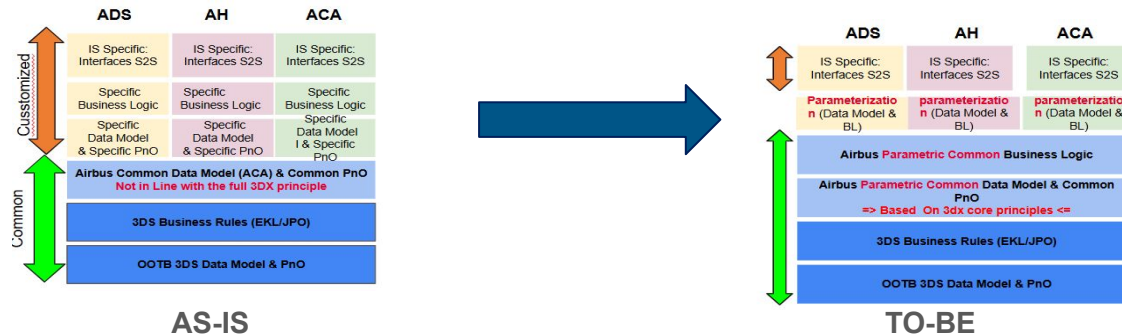
Context and Objectives

Context

- 3DX One Platform delivers business capabilities across Airbus Divisions (BU)
- Data model and business behaviour (business logic i.e BL) are the main enablers used to build the capabilities.
- Across the years, due to various constraints (Business, IM), the specific BU part has increased, generating a huge number of artefacts to maintain

Objectives

- Provide a new design pattern to increase the communality part of the solution



Benefits

- Reduce number of technical artifacts to be maintained
- Enable Business Unit Specific behavior adoption via parameterization
- Reduce code duplication

Data Model Principles (Reminder)

 DRIVERS

From a business and end user point of view, the capability to add customized data model information is key.

Deployment extensions (DEXT) enable the administrator to add attributes to existing DS types depending on the collaborative space you belong to.

- Recommended when an automatic data model addition process is required (totally or partially automatic, depending on the CS or on another criteria)
- Recommended when it is required to **not remove** this data model
- Recommended for objects piloted by interfaces
- Properties layout: attributes are in the same frame than OOTB attributes
- CEXT behavior **can not** be reproduced

Customer extensions (CEXT) enable a end user to add or remove attributes on demand.

- Recommended when the users want a flexibility
- Not recommended for objects piloted by interfaces
- Drawback: the user needs to have a good knowledge of the data model
- The data model is not necessarily harmonized to all objects with same type/usage
- Properties layout: attributes are in a specific section
- DEXT behavior can be reproduced (with BL usage)

Common for DEXT or CEXT:

- Need to smartly manage the granularity of attributes grouped together
- Capability to manage edition is equivalent
- Capability to select attributes (filtering, web customized views, etc...) is equivalent

Rationale on Extensions usage

Objectives

5/7/2025
This slide is to present to you in a few words the reasons which motivated the needs related to Extensions usage within 3DX One Platform Data Model

Rational of Specialisations

Objectives

This slide is to present to you in a few words the reasons which motivated the need for specialization of objects in 3DX One Platform

DRIVERS

From a business and end user point of view, we generally need as many business objects as platform objects to carry out the activities of the different domains.

After observing a limited number of standard objects in the platform compared to the specificities of the Business object needs, the proposed solution was to proceed with the specialization of the objects (requirement, physical product.... etc) in order to extend the capacity of the tool to address business requirements mainly on the aspects below:

- Facilitate object search functionality by domain (without necessarily using 6WTag)
 - Process lighter than managing attribute
- Manage the categorization of object families
 - Manage deployment extensions on specialized types
 - Attribute based typing implies to have a BL looking to the attribute value to decide to add an extension or not.
- Manage distinct behavior on objects
 - High Level OOTB settings applicable to some object types only (example Change Action promotion as per SC role, revise of VPMRef can be done only from a certain Le status)
- Manage ergonomics and element display info (icons, type label, advanced search, type explanatory appearing in apps in addition to ootb one ...)
- Type not changing among Versions Upgrade (no change in ergonomics when OOTB type changes). Example for change action management when changing from “change action apps” to “change governance/execution apps”. If this was managed through predefined dashboard views we had to redo everything when jumping to the new app (having business type in a columns).

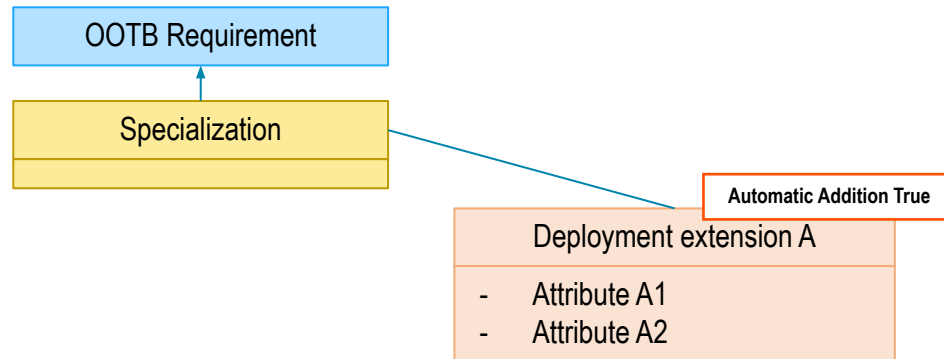
Note: Fine object management was not sufficient via attributes such as Business Type (BT)
For details on One Platform referential ref. to: [file](#) (“Specializations” tab)

CONTEXT TO USE A SPECIALIZATION

- If we want to add attributes on an object not allowing usage of Customer Extension (CEXT) or Deployment Extension (DEXT)
- Require a dedicated icon (but not sufficient)
- Require a segregation with OOTB object
- Require a dedicated behavior

Design Pattern:

 If you want to add attribute to a specialization. We recommended to not add it directly the specialization. Instead of create a Deployment Extension automatic addition true and add attributes belonging it. (On Premise)



CONTEXT TO USE A DEPLOYEMENT EXTENSION

- To add a pool of attributes on object type allowing usage of Deployment Extension (DEXT)
- To reuse a pool of attributes between different objects (Objects in the same family (ER or IRPC))
- To don't be able to remove this pool of attributes
- To facetize automatically an object depending of criterias (CS, attribute,...) with help of a BL

Design Pattern:

-  The pool of attributes has to be defined to be as much as possible reusable by other object
-  As much as possible DEXT should be Automatic addition true => This allowing to not use BL

Deployment extension A

- Attribute A1
- Attribute A2

CONTEXT TO USE A CUSTOMER EXTENSION

- If we want to add a pool of attributes driven by a customer decision. Usage of Customer Extension is recommended
- If we want to add a pool of attributes on object type allowing usage of Customer Extension (CEXT)
- If we want to reuse a pool of attributes between different objects (Objects in the same family (ER or IRPC))

Parametric Data Model Design Rules

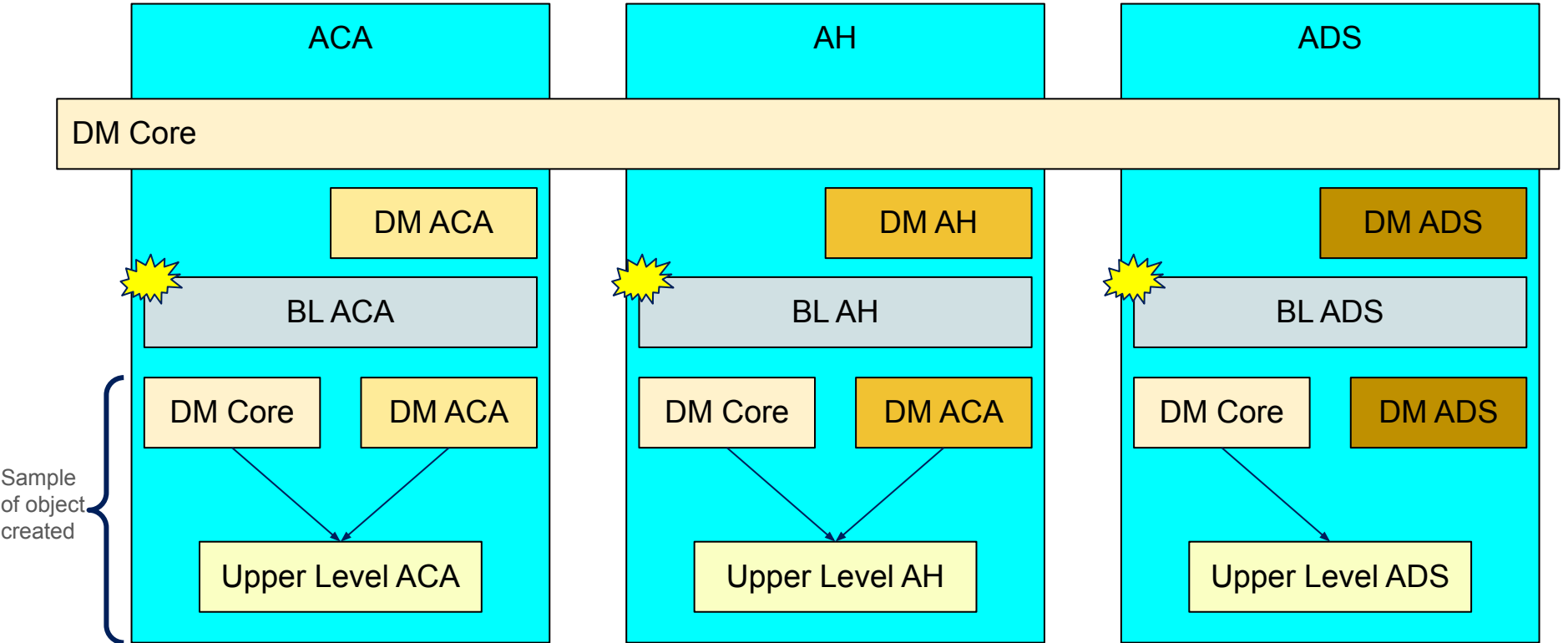
Rule ID	Description	Applicable for artefact
PRDM_1	Data Model instantiation BL have to be - Merged in a single BL per Factype & OpenID - Binded only in the CS-Corporate	Client BL, Server BL, Precomit BL <i>TBC for behaviour BL</i>
PRDM_2	An extensions with auto add option will be defined only when the extension is applicable for ALL BU/Programme	Data model
PRDM_3	Resource file to be used in order to list the extensions applicable for an Object. The key to get the value should be set in a manner that a generic code can be used in the BL to get the right value	Client BL <i>ER opened to request it also on server/precomit BL</i>
PRDM_4	BL code should match with the format defined in this document.	Client BL, Server BL, Precomit BL

Parametric Data Model Design Rules

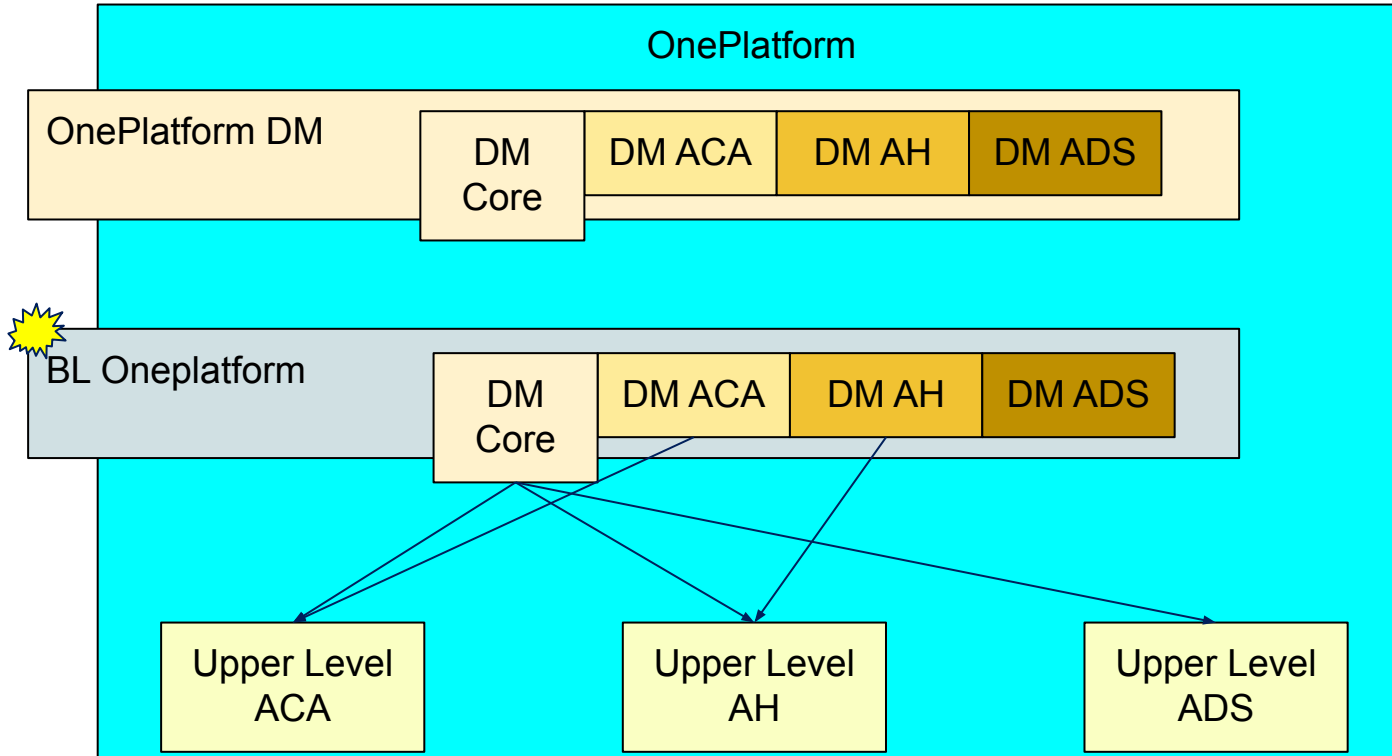
Rule ID	Description	Applicable for artefact
PRDM_5	Awaiting the ER (COTS-10579), the <i>parameter section</i> of the BL code will contain all parameter and values to be used in order to list data model applicable or behaviour activation for an Object. The key to get the value should be set in a manner that a generic code can be used in the BL to get the right value	Server BL, Precommit BL
PRDM_6	Duplication of code in the same BL is not allowed	Client BL, Server BL, Precommit BL
PRDM_7	The layout will be done with the parametrization that give the right layout based on the object and its context	Client BL, Server BL, Precommit BL
PRDM_8	A behaviour inside a BL should be activable/deactivable.	Client BL, Server BL, Precommit BL

Move to new Design Pattern

Move to new Design Pattern - AS IS



Move to new Design Pattern - To Be (1/2)



Main rules

- Same Data Model & BL deployed all BUs
- Specific behaviour managed by parameter
- BL merged for each object type and OpenID
- Common behaviour not duplicated

Move to new Design Pattern - To Be (2/2)

Organisation - AIRBUS_AND_EXT_ENTERPRISE

Organisation - Airbus_Group

CS -
Standard Component

Standard Part

Standard Part

Standard Part

Organisation - ACA

Organisation - AH

Organisation - ACS

CS - A350

Upper Level ACA

CS - H175

Upper Level AH

CS - Doc
Management

Doc ACA

Doc AH

Doc ADS

Main rules

- Specific behaviour will be implemented based on PnO rules
→ Mix of Collaborative space & organisation

Move to new Design Pattern - Process to merge Business Logic

Rule:

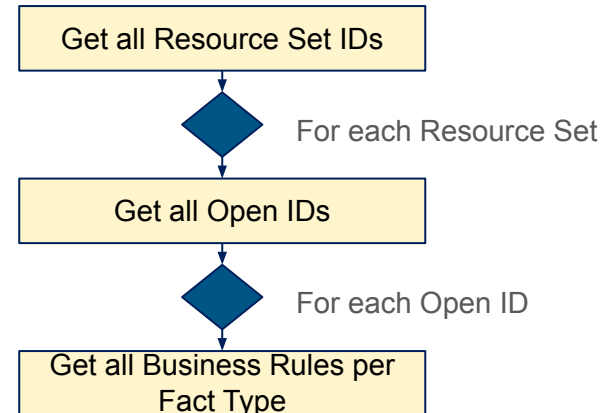
 A datamodel instanciated into the same Security Context (Intersection between Organization and Collaborative Space) must have the same extensions.

Assumption:

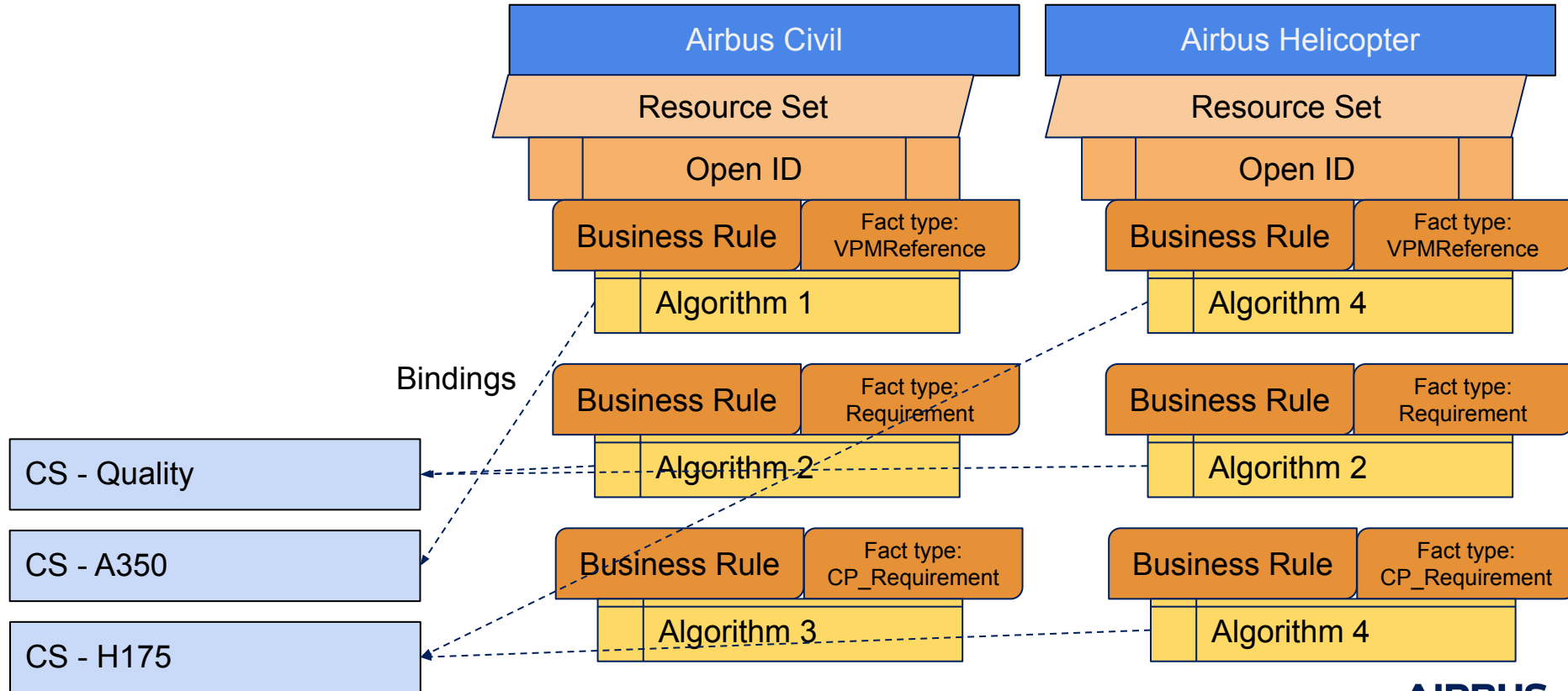
In order to simplify business logic management. We want to deploy the same BL on the 3 BU.

To do that, the easiest way is to merge Business Logic by following the process below:

- Identify BL having the same Resource Set ID.
- Into each of them, identify BL having the same Open ID
- Into each Open ID, identify Business Rule having the same fact type



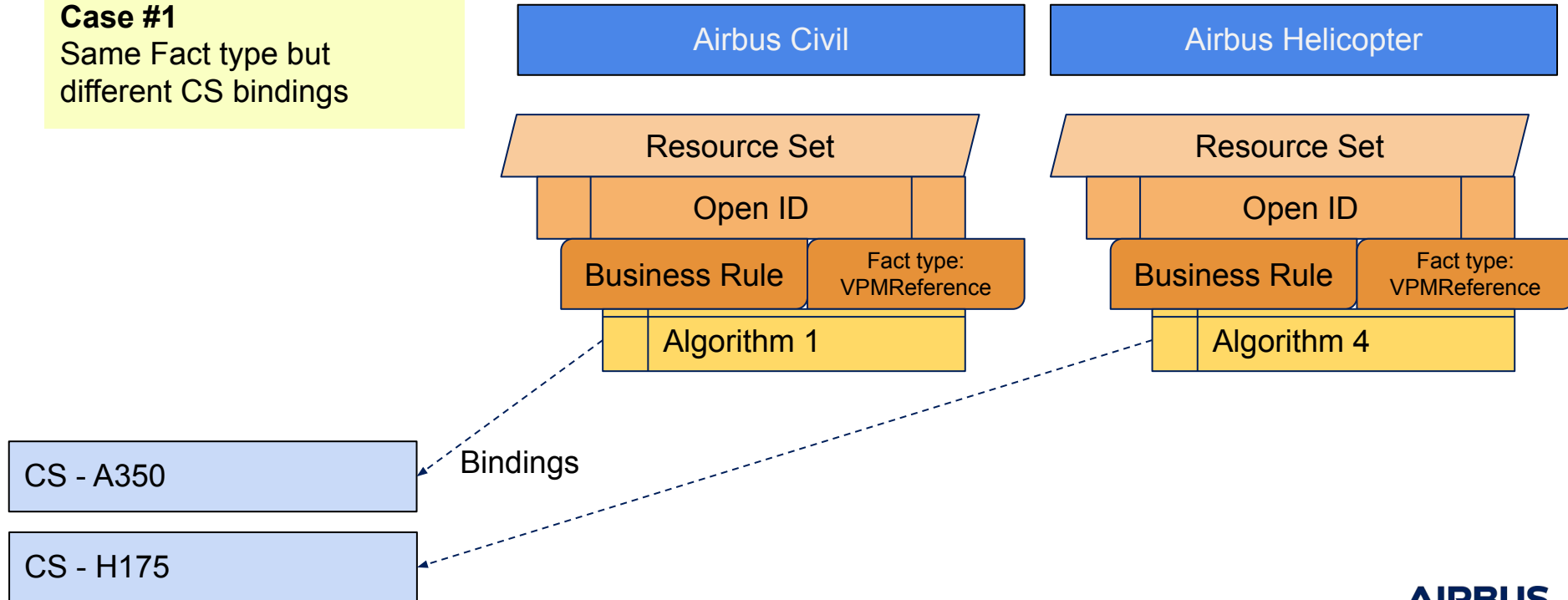
Rules to merge datamodel instantiation BL (As-Is)



Rules to merge datamodel instantiation BL (To-Be)

Case #1

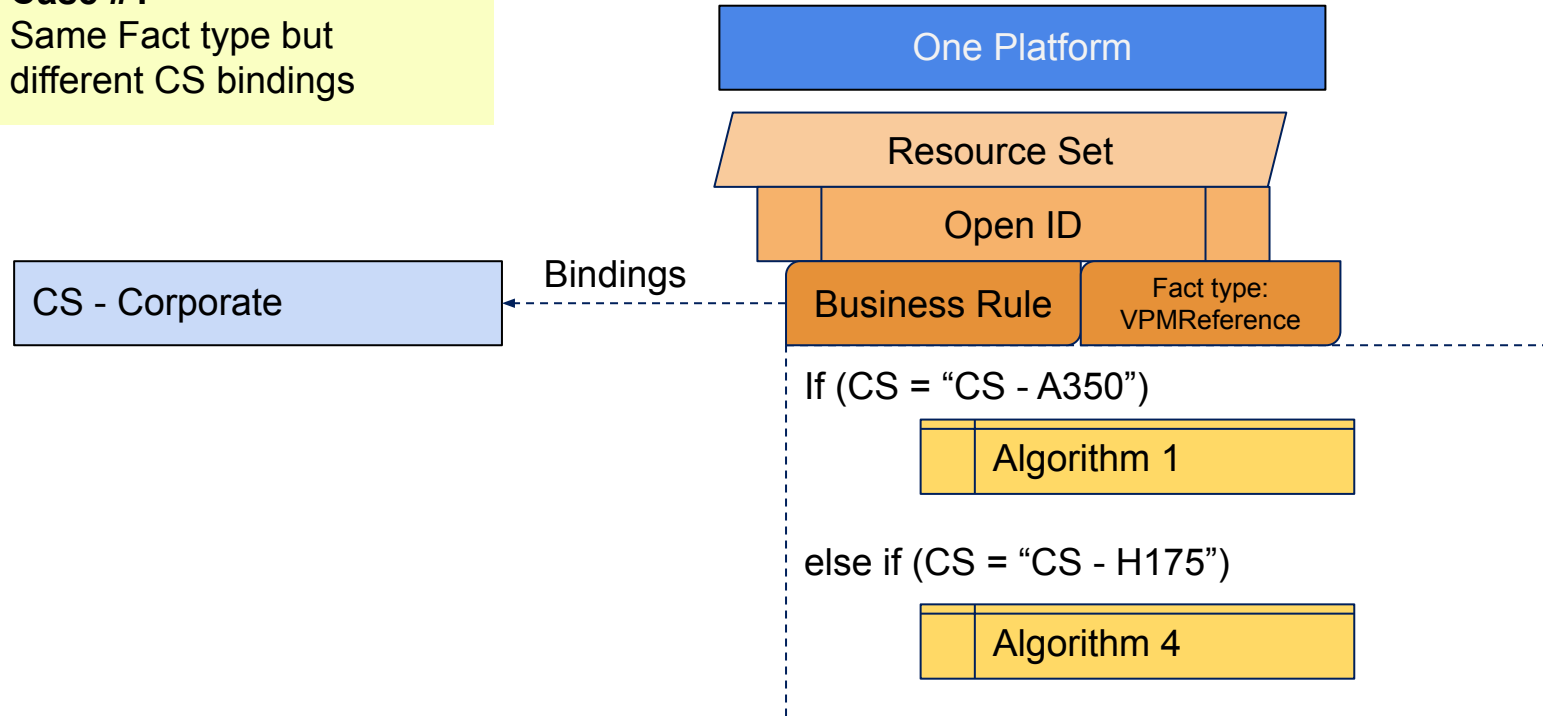
Same Fact type but different CS bindings



Rules to merge datamodel instantiation BL (To-Be)

Case #1

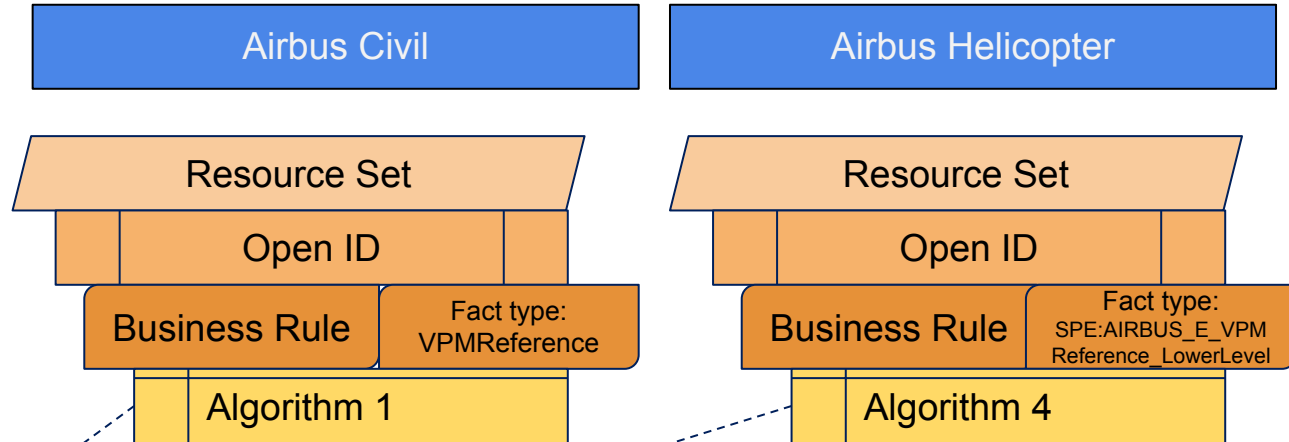
Same Fact type but
different CS bindings



Rules to merge datamodel instantiation BL (To-Be)

Case #2

Same Fact type or its specialization but different CS bindings



CS - A350

CS - H175

Bindings

Rules to merge datamodel instantiation BL (To-Be)

Case #2

Same Fact type or its specialization but different CS bindings

CS - Corporate

Bindings

One Platform

Resource Set

Open ID

Business Rule

Fact type:
VPMReference

If (CS = "CS - H1750" && type=SPE:AIRBUS_E_VPMReference_LowerLevel)

Algorithm 4

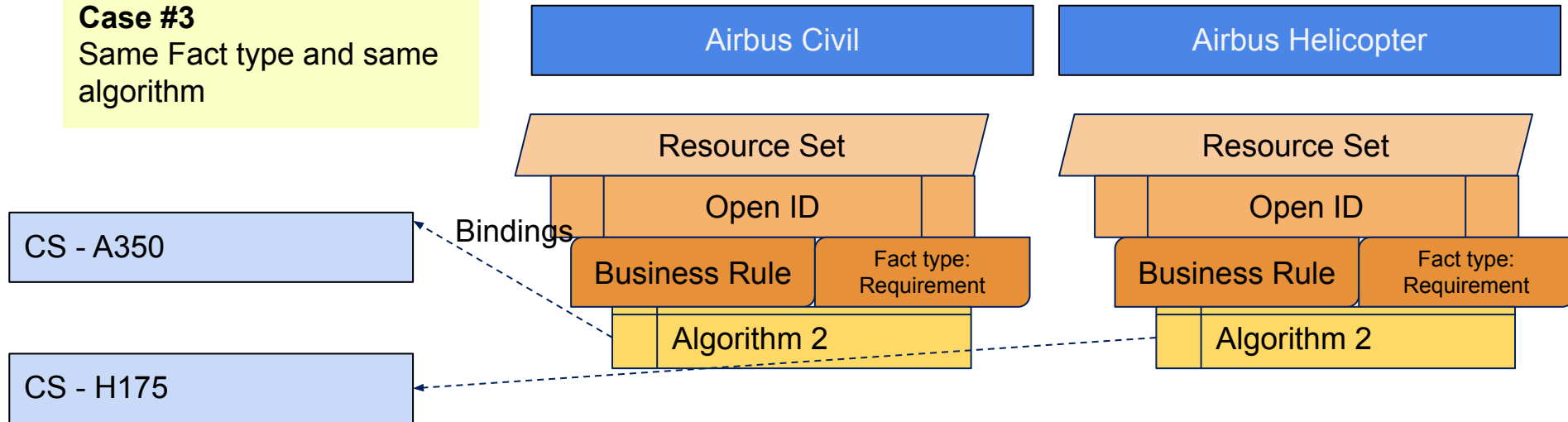
else if (CS = "CS - A350" && type=VPMReference)

Algorithm 1

Rules to merge datamodel instantiation BL (To-Be)

Case #3

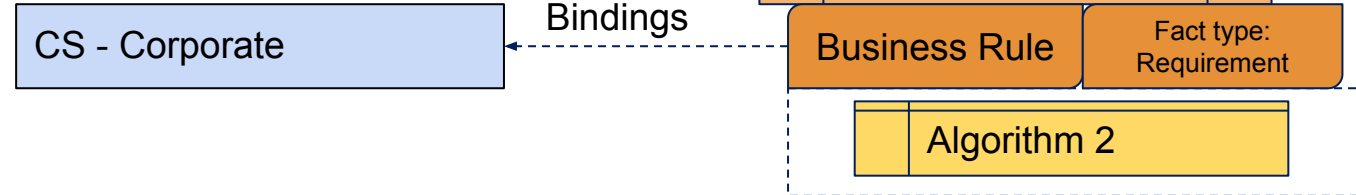
Same Fact type and same algorithm



Rules to merge datamodel instantiation BL (To-Be)

Case #3

Same Fact type and same algorithm



Design Pattern BL implementation - BL Format

Description Section

Parametrization section CS / Orga / Program.. Prefixed by the FCType and OpenID

Common section - Applicable whatever the context / BU/ Programme Repeat each time we have a common section

Specific section can be activated by parameterization Repeat each time we have a common section

```

/*DescriptionStart
Team:
OpenID:
Factype:
Input:
output:
Purpose:
DescriptionEnd*/

/*ParametrizationStart */
FCTYPE_OPENID_Param = "Value1"
...
/*ParametrizationEnd */

/*CommonStart*/
if(ThisObject.HasAttribute("FCTYPE_OPENID_Param"))
{
    strBusinessType = ThisObject.GetAttributeString("FCTYPE_OPENID_Param2")
    Trace(1,"BT : " + strBusinessType)
    Trace(1,"Type : " + ThisObject.PrimaryType.Name)
}
/*CommonEnd*/

/*SpecificStart*/
if(ThisObject.HasAttribute("FCTYPE_OPENID_Param"))
{
    strBusinessType = ThisObject.GetAttributeString("FCTYPE_OPENID_Param2")
    Trace(1,"BT : " + strBusinessType)
    Trace(1,"Type : " + ThisObject.PrimaryType.Name)
}
/*SpecificEnd*/

```

Main Rules

- Parameter named with **FCType_OpenID** as a prefix
- No code duplication
- On BL client, the parameter is managed in a dedicated Ressource File

Design Pattern BL implementation - active/deactive behaviour

Parametrization section CS / Orga / Program..

Prefixed by the FCType and OpenID
List all rules applicable to each
combination object/CS/ORG

Get the list of rules
applicable

Apply the rule if
applicable

```

/*ParametrizationStart */
FCTYPE_OPENID_<CS1.ORG1> = "Rule_1,Rule_2,...Rule_x"
FCTYPE_OPENID_<CS2.ORG2> = "Rule_2,Rule_5"
...

/*ParametrizationEnd */

ListOfRules=checkRulesActivated(This)

If (Rule_1 in ListOfRules)
If (Rule_2 in ListOfRules)
...
If (Rule_n in ListOfRules)
  
```

Main Rules

- Each rule should be activable/deactivable
- No code duplication
- Activation could be based on pno or other criteria to be implemented

Value improvement - Technical Artefacts reduction

Artefacts reduction with implementation of new pattern (if same strategy is done for behaviour BL too):

AS-IS

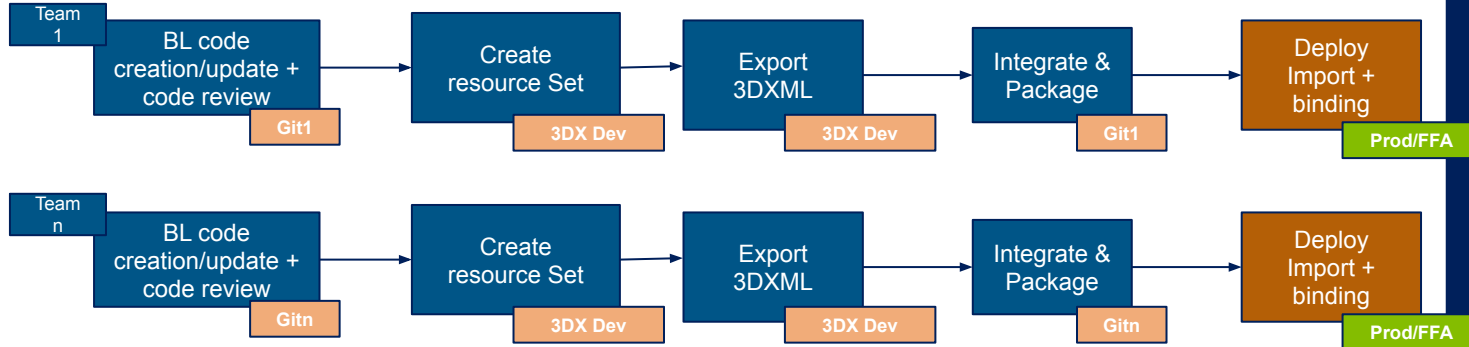
BU	COUNTUNIQUE
	0
ACA	630
ADS	371
AH	189
cross_BU	430
Grand Total	1464

TO-Be

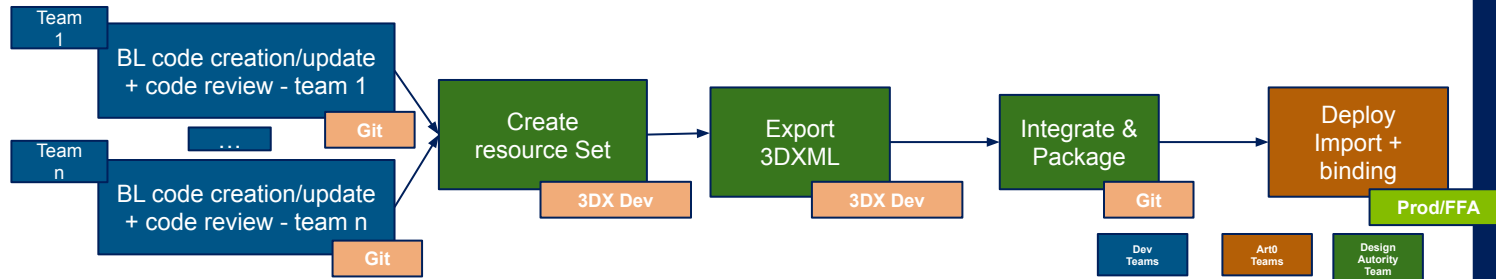
Grand Total	97
--------------------	-----------

Technical artefacts governance

AS-IS Model: Each team develop/maintain and generate package containing the BL of its perimeter.



TO-BE Model: All teams will work on the same project to develop & review BL code. Ressource set and package creation and delivery to be done by a central team (Design Authority).



New BL Delivery Process

- Same GIT project used by all teams
- Resource generation is done by Design Authority
- Packaging and delivery Design Authority

Limitations

- Rationalisation of data model and business logic related to it for OnPremises only
- SaaS impacts is not studied into this document
- Data model convergence impacts not fully assessed
- Performance impact to be checked later
- Securisation the code

End